COSC204 Computer Systems

Lecture 15: File Systems

Zhiyi Huang

School of Computing, University of Otago

Overview

Last lecture:

- How operating systems manage processes
- System calls
- Interrupt handling
- Life cycle of processes

This lecture: file systems.

- What's a file?
- File access patterns
- Directory structure
- File system implementation
- Disk allocation methods

Motivation for the concept of files

The file system is an important interface between the OS and users.

Often, programmers do not need to be aware about the device or media on which data is stored.

- They can focus on the data itself, and how to get at it.
- The challenge of managing *how* data is stored can be left to the operating system.

The operating system provides a *logical unit of storage* for the user, called a file.

- Files are thus an abstraction between the user and the OS.
- The user / developer makes reference to data in files.
- The OS maps file data onto regions of underlying storage.

Defining the concept of a file

What does a file hold? A collection of *related* data, e.g., the set of lines in a program, the set of words in a text document.

What structure does a file have?
A byte stream or a sequence of bytes. A 'text file' is a stream of character codes (e.g., historically ASCII, now UTF-8, or . . .).

What can a user do with a file?
Operations such as: create, write, read, reposition, delete, truncate. (These are all achieved by making system calls.)

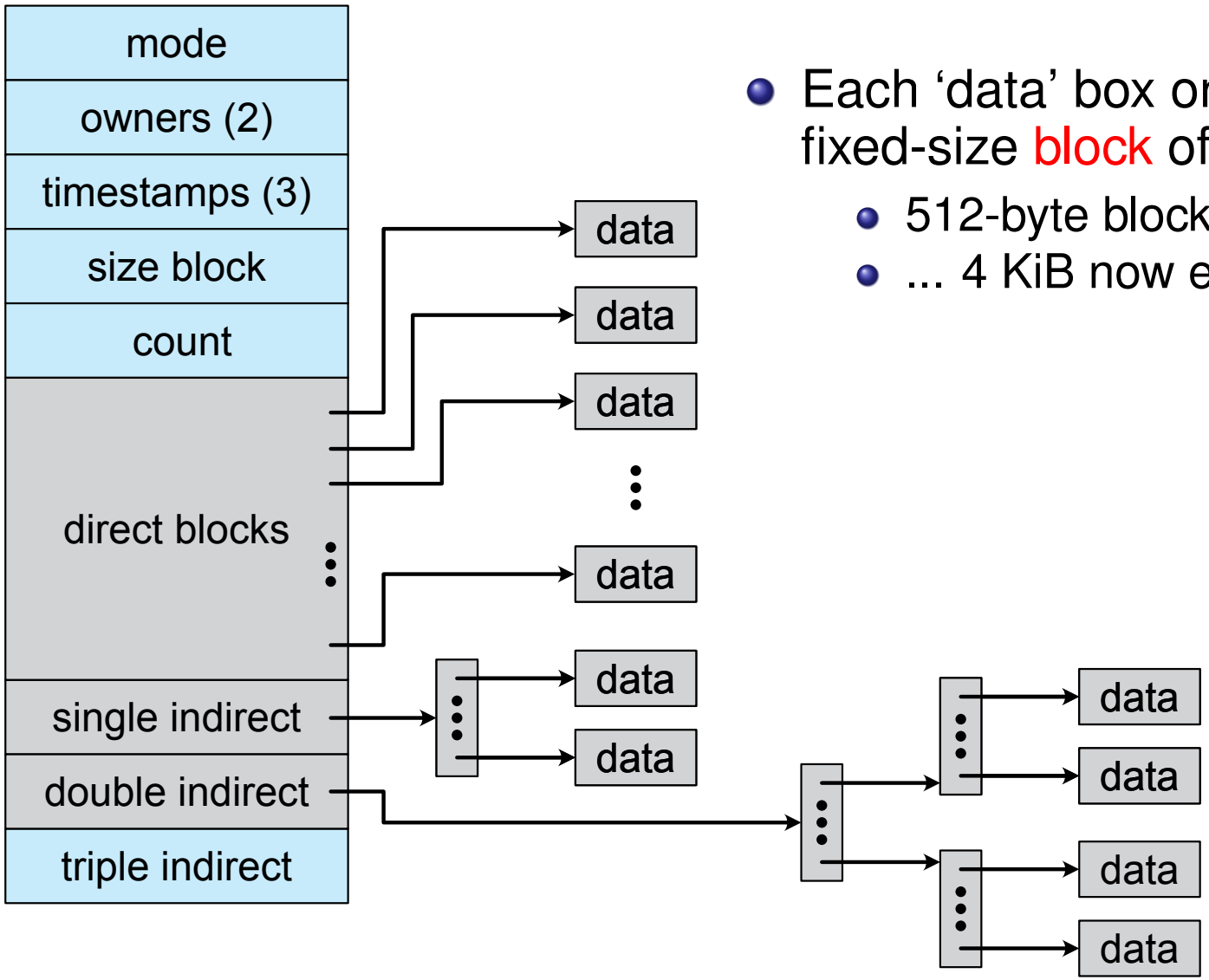
Files have attributes as well as data

Besides the data *in* the file, the operating system needs to store data *about* each file in the file system.

- File name
- Owner
- File type
- Location of data
- Size of data
- Access permission
- Housekeeping information—e.g., timestamps

Q: In a typical Linux context, where is all this information kept?
A: In an index node (inode) pointed to by an entry of a directory.
(Directories are essentially just a sets of (name, inode) entries.)

Index nodes (inode) show where data blocks of file are



- Each ‘data’ box on the figure is a fixed-size block of disk storage.
  - 512-byte blocks were common,
  - ... 4 KiB now emerging.

System calls relevant to file systems

- Key system calls include `open`, `close`, `read`, `write`, and `ioctl`.
- `ssize_t write(int fd, const void *buf, size_t count);`
  - Write up to `count` bytes from the buffer pointed `buf` to the file referred to by the file descriptor `fd` at the current file offset.
  - The return value is the number of bytes written on success. The number of bytes written may be less than `count` if, for example, there is insufficient space on the underlying physical medium.
  - On error, `write()` returns -1 and `errno` is set to the error code, e.g.: `EAGAIN`, `EFAULT`, `EINTR`, ...
  - The return value of any system call must be always checked in user applications and errors must be dealt with properly in order to guarantee reliability and robustness.
- `ssize_t read(int fd, void *buf, size_t count);`
  - Attempt to read up to `count` bytes from file descriptor `fd` into the buffer starting at `buf`. On success, the number of bytes read is returned and the file position is advanced by this number. It may be less than `count`. Zero is returned if the end of the file is reached.
  - On error, `read()` returns `-1` and `errno` is set to the error code.

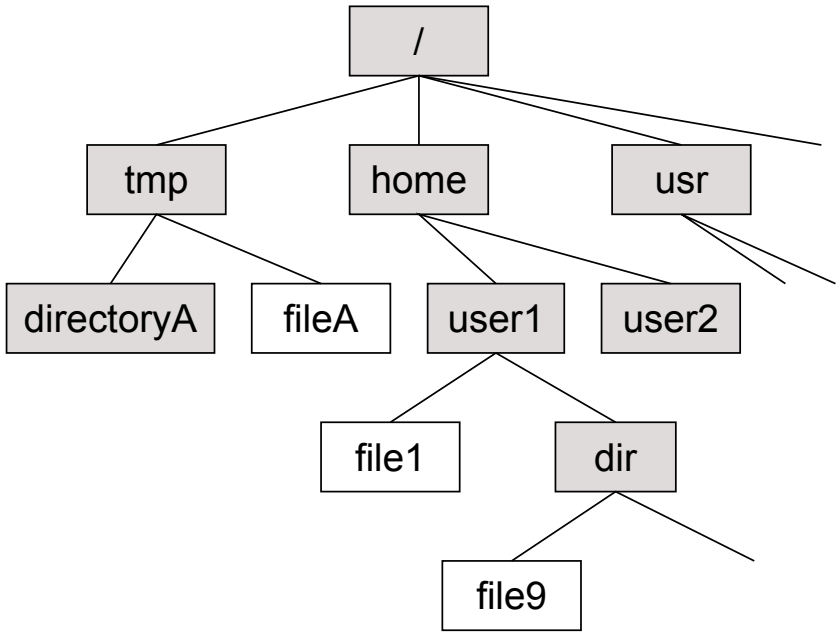
Applications often have particular file access patterns

Two common file access patterns used by applications:

- Sequential access: (a very common approach)
  - A file pointer identifies a record within the file.
  - The pointer can be moved incrementally forwards (in read or write operations) or backwards (in rewinding).
  - The hardware analogy is a tape.
- Direct access: (also known as random access)
  - A file is viewed as a numbered sequence of records.
  - Operations (e.g., read, write) can be carried out on any record in any order.
  - The hardware analogy is a disk (e.g., spinning media, or SSD).

Directories can contain directories: tree-structured

A directory is a table of tuples, and each each tuple is a pair of a file name, and a file pointer. Each directory can have sub-directories, forming a tree structure. (Usual upside-down CS trees...)



To refer to an arbitrary file in the tree-structure, we now need to specify a path from the root of the tree, e.g., `/home/user1/dir/file9`

Implementing management of files: a file system

So far, we have been describing the logical structure of the file system—files and directories.

The operating system has to map this logical structure onto a storage device (typically, a disk).

This is done by the file organisation module.

- We have already seen how the smallest unit of allocation in a disk is a block.
- The file organisation module has to *allocate* blocks for the storage of files.
- A filesystem often has its own concept of blocks that we call logical blocks: these help the filesystem manage actual disk blocks.

Disk allocation methods: congtiguous allocation

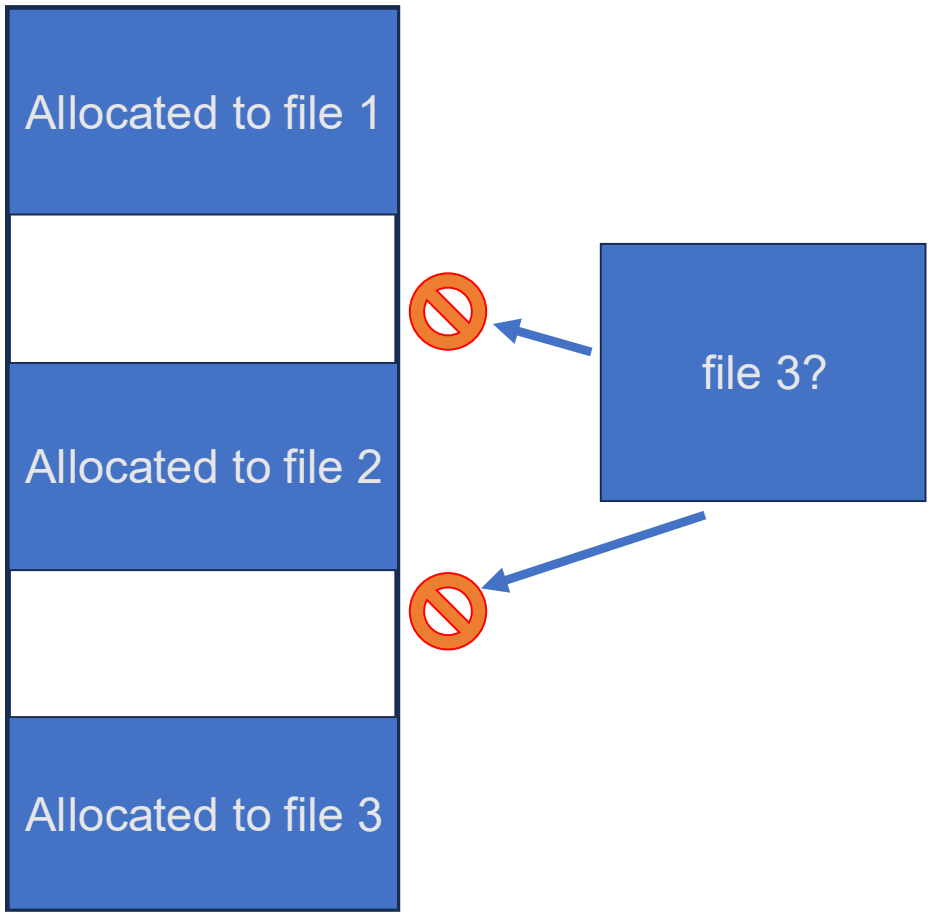
How should we set aside disk space for files to occupy?

There are several allocation methods.

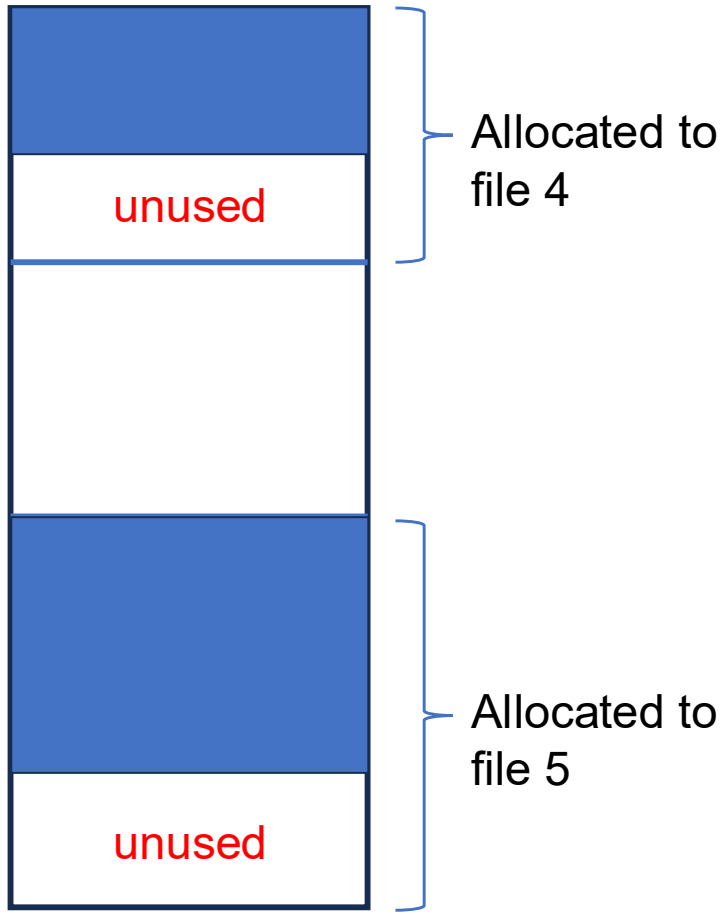
Contiguous allocation

- Each file occupies a set of contiguous blocks on the disk.
- Advantages:
  - Sequential access? Works well, because the next character to read is in the current block, or the very next block.
  - Direct access? Works well, just count the number of blocks.
- Disadvantages?
  - External fragmentation: a situation where the free space is fragmented. The total free space could fit the file but none of the contiguous spaces are large enough to do so.
  - Files can "grow", and we don't know how big a file will become. Even if we did, files can shrink too, in which case there would be internal fragmentation, where some space is allocated but not used.

Depicting internal and external fragmentation



External fragmentation



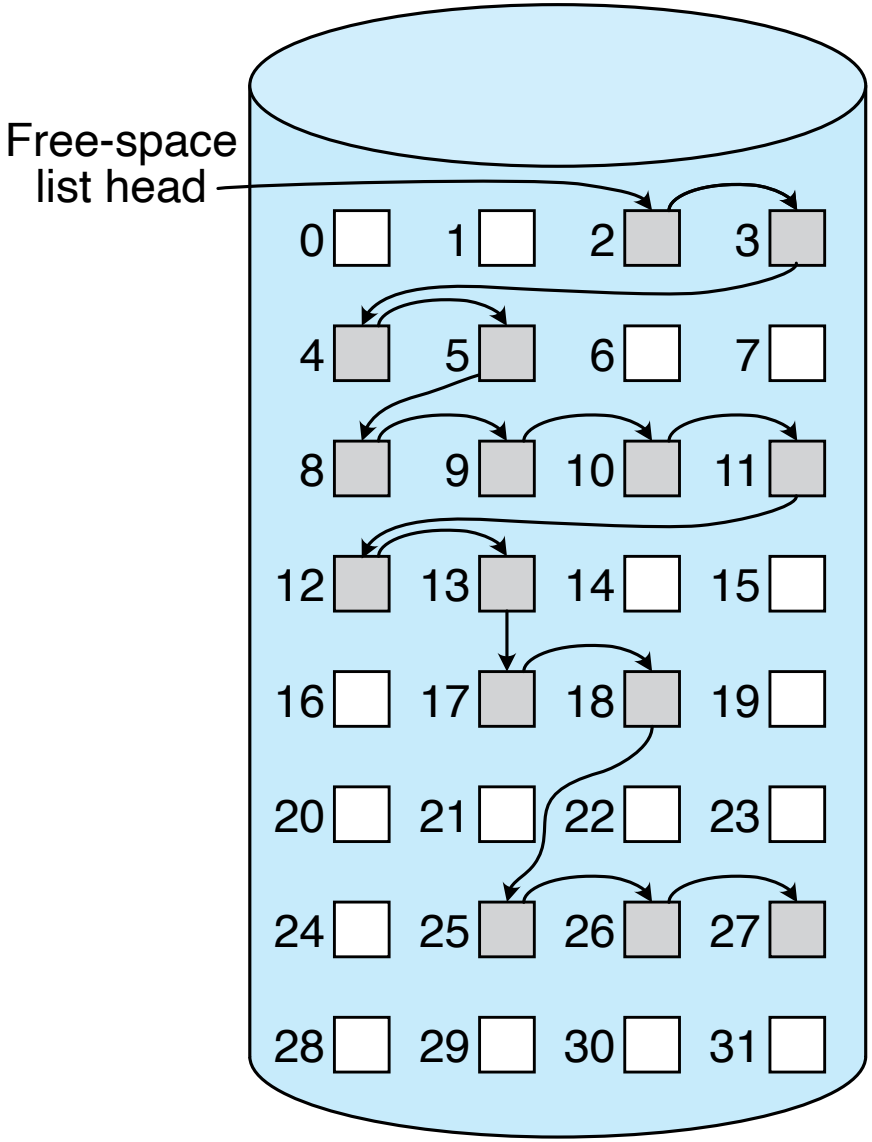
Internal fragmentation

Disk allocation methods: linked allocation

Linked allocation

- Each file is a *linked list* of disk blocks.
- The directory entry contains a pointer to the first (and last) blocks of the file.
- Each block of the file contains a pointer to the next block.
- Advantages?
  - No external fragmentation.
  - Files can be arbitrarily big; no need to pre-allocate.
- Disadvantages?
  - Can only easily be used to support sequential access patterns.
  - Pointers take up some space. (This can be addressed by *clustering* blocks into larger allocation chunks.)
  - Internal fragmentation. (Which gets worse the larger your clusters are.)
  - Reliability. If one pointer is lost, the whole file thereafter will be too.

Unallocated blocks are also tracked using a linked list



Disk allocation methods: File Allocation Table (FAT)

File Allocation Table (FAT)

A variant of linked allocation, used in MS-DOS, and Windows.

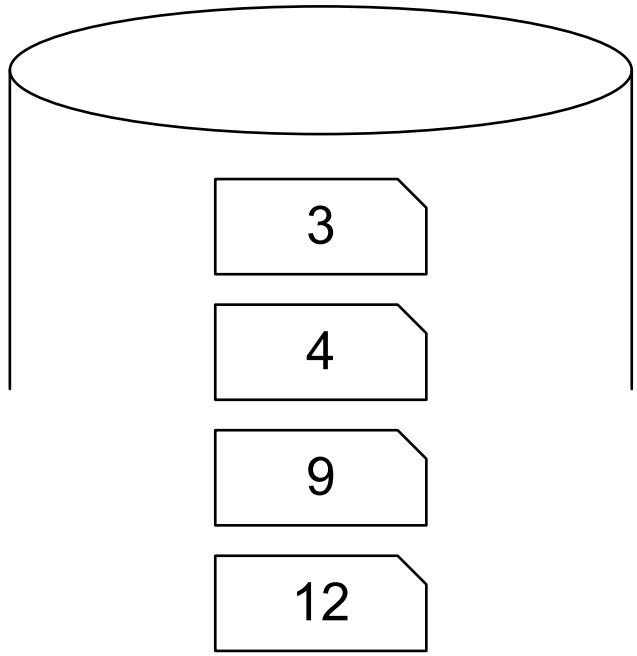
- A table is created at the beginning of each disk (actually the partition, but we've not talked about partitions), with an entry for every block in the file system.
- The directory entry for a file specifies the block number for the first block of the file.
- The value of the FAT entry for the first block will identify the block number of the next block in the file.
- Advantages? Same as linked. In addition, direct access is better supported, because chaining through the FAT is faster than chaining through a linked list stored across data blocks.
- Disadvantages? Same as linked. Even more disk head seeks: i.e., to and from the FAT. Can speed up by caching the FAT.

Disk allocation methods: depicting FAT allocation

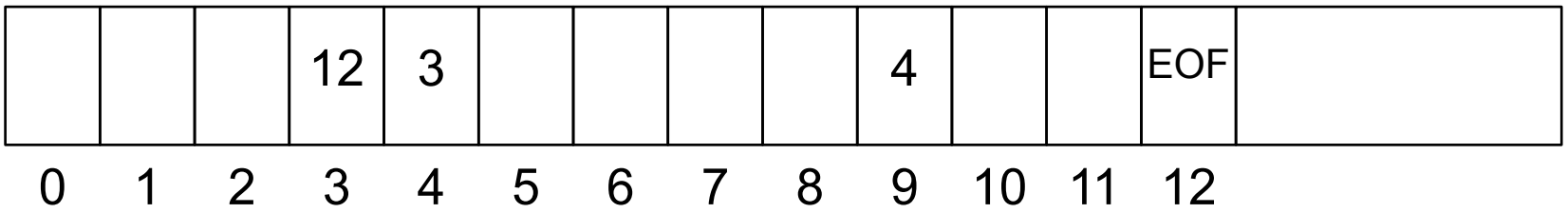
File Allocation Table (FAT)

Directory entry for the file

Name: fname
First block: 9



Entries for ‘fname’ in the FAT

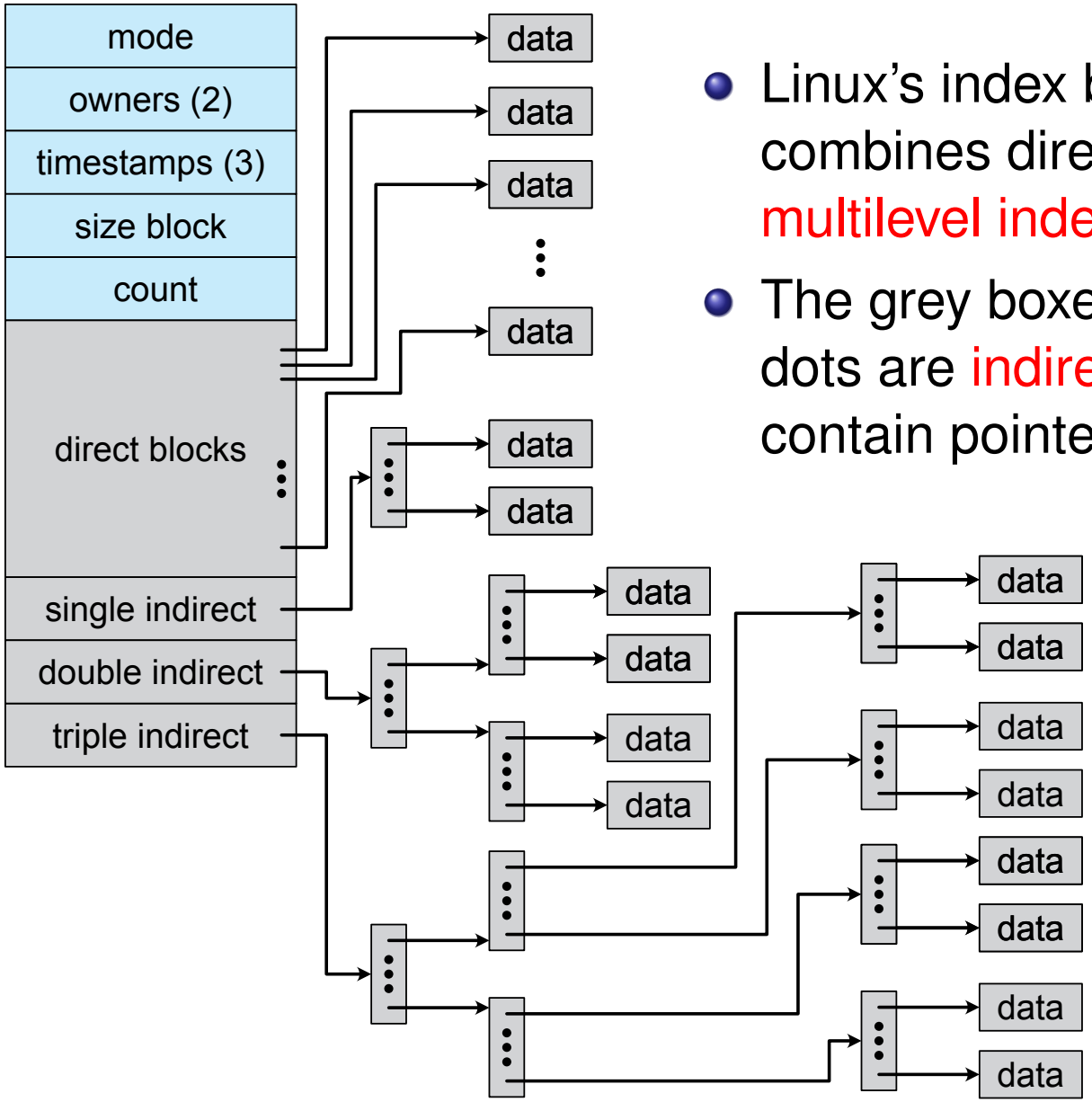


Disk allocation methods: indexed allocation

Indexed allocation

- Each file has an index block (inode), containing a table specifying the physical block for each logical block.
- The directory entry for a file contains the address of its index node.
- Advantages?
  - Very direct access.
  - No external fragmentation.
- Disadvantages?
  - Wasted space—internal fragmentation, really. We have to allocate a larger array for the index, because we don't know how big the index needs to be. Also lots of head seeks, like linked allocation.

Disk allocation methods: depicting indexed allocation



- Linux's index block (inode), combines direct indexing with multilevel indexing.
- The grey boxes containing links and dots are indirect blocks that just contain pointers to other blocks.

Readings

For this lecture, relevant material is Chapter 10 (Sections 1–6) and Chapter 11 (Sections 1–4).